

Trabajo Práctico de Implementación: Esteganografía

Criptografía y Seguridad (72.04) — Instituto Tecnológico de Buenos Aires

Lucas Di Candia

29 de septiembre de 2026

Índice

Introducción	3
Cuestión I: Análisis del documento de Majeed y Sulaiman	3
Organización formal del documento	4
Descripción del algoritmo	5
Notación: claridad, errores y contradicciones	7
Conclusión	7
Cuestión II: Comparación de LSB1, LSB4 y LSBI	7
Metodología	10
Resultados	10
Análisis	12
Cuadro comparativo	13
Conclusión de la comparación	13
Cuestión III: Procedimiento de estegoanálisis	14
Cuestión IV: Mensajes ocultos dentro de mensajes ocultos	14
Cuestión V: El fragmento de película	14
Cuestión VI: El método de ocultamiento desconocido	14
Cuestión VII: Por qué LSBI mejora a LSB	14
El mecanismo	15
Evidencia medida	15
¿Es realmente una mejora?	16
Cuestión VIII: Otras formas de guardar los patrones invertidos	17
La elección actual	17
Alternativas	17
Análisis de las alternativas principales	19
Preferencia	19
Cuestión IX: Dificultades de implementación	20
El algoritmo LSBI	20
Encipción	21

Formato del mensaje y portador	22
La consigna y los archivos de ejemplo	22
Compilación y portabilidad	23
Cómo se verificó	23
Cuestión X: Mejoras y extensiones futuras	24

Introducción

stegobmp es un programa de línea de comandos escrito en C, con OpenSSL, que oculta cualquier archivo dentro de una imagen BMP de 24 bits sin compresión (versión 3) y lo recupera después. Implementa los tres métodos que pide la cátedra [2]: LSB1, LSB4 y LSBI, este último la variante con inversión de bits de Majeed y Sulaiman [1]. Opcionalmente, antes de ocultar, cifra el mensaje con AES (128, 192 o 256 bits) o con 3DES, en los modos ECB, CFB, OFB o CBC, derivando la clave y el vector de inicialización con PBKDF2 a partir de una contraseña.

La esteganografía oculta la existencia misma del mensaje, a diferencia de la criptografía, que oculta su contenido [3, 15]; por eso el programa permite combinar ambas.

Organización. El informe responde en orden las cuestiones de la sección 7 del enunciado, una sección por cuestión, y cada respuesta empieza transcribiendo la pregunta. Las Cuestiones I y II analizan el documento de Majeed y Sulaiman y comparan los tres métodos; las Cuestiones VII y VIII explican y discuten la mejora que propone LSBI; la IX cuenta las dificultades de la implementación y la X propone mejoras. Las Cuestiones III a VI dependen de los archivos de estegoanálisis de la cátedra, que todavía no fueron entregados, y quedan marcadas como pendientes.

Convenciones. Las páginas citadas del documento de Majeed y Sulaiman son las de la revista (342 a 348). Todas las cifras medidas que se citan en las Cuestiones II y VII (capacidades, bytes modificados, MSE, PSNR, estadísticos χ^2 , conteos de inversión) se regeneran con **make report** a partir del propio programa y de las herramientas de análisis del repositorio, y se contrastaron con los archivos de **Ejemplo/** de la cátedra; ninguna se escribió a mano. Las rutas como **docs/LSBI-NOTES.md** remiten a las notas del proyecto. Para regenerar este informe basta ejecutar **make report** en la raíz del repositorio.

Cuestión I: Análisis del documento de Majeed y Sulaiman

Enunciado. *Discutir los siguientes aspectos relativos al documento. a) Organización formal del documento. b) La descripción del algoritmo. c) La notación utilizada, ¿es clara? ¿hay algún error o contradicción?*

El documento [1] propone una variante de LSB para imágenes de 24 bits. Primero reemplaza los bits menos significativos con los bits del mensaje, como en LSB común. Después clasifica los bytes de color según el patrón que forman el segundo y el tercer bit contando desde el menos significativo (“the 2nd last and 3rd last bit”, p. 344), cuenta en cada patrón cuántos bytes cambiaron y cuántos no, y si cambiaron más de los que no, invierte el bit menos significativo de los bytes de ese patrón. Sólo se usan los canales verde y azul (p. 342) y el estado de cada patrón se anota en un registro cuyo lugar el documento nunca define (“specific location”, p. 344). La consigna de la cátedra [2] completa ese hueco: fija que el registro son cuatro banderas guardadas con LSB1 en los primeros cuatro bytes del bloque de píxeles, fija el orden físico de los bytes (azul, verde, rojo) y el tamaño en big endian. Por eso conviene separar lo que dice el documento de lo que agrega la consigna, y este análisis lo hace en cada punto.

Cada error o contradicción que se le atribuye al documento se relevó en el texto del PDF o en una imagen de la página, y va con su página. Los juicios que son una valoración propia, y no un error verificable, se formulan como tales (“consideramos”, “a nuestro juicio”).

Organización formal del documento

El documento tiene ocho secciones: introducción, la técnica LSB estándar, el modelo de color RGB, otra vez LSB, la técnica propuesta, el algoritmo propuesto, los resultados y una discusión con conclusión. La primera falla de organización se ve en la página 343. La introducción termina con la frase “This paper is organized as follows:” y el encabezado de la Sección 2 aparece de inmediato, de modo que el esquema que la frase anuncia nunca se escribe. Consideramos además que la introducción dedica buena parte de su extensión a la etimología de “steganography” y de “cryptography” (pp. 342–343) y que la propuesta recién se enuncia en las últimas líneas de la página 343, sin plantear antes el problema concreto que se quiere resolver.

Hay material repetido. La Sección 2 (p. 343) describe la técnica LSB estándar y sus debilidades, y la Sección 4 (p. 344) vuelve a describirla con un ejemplo y un pseudocódigo. La Sección 3 (p. 343) presenta el cubo RGB y, a nuestro juicio, no aporta nada al método, porque nunca se vuelve a usar; además contiene dos errores de notación que se detallan en el cuadro 2. El método propuesto se describe dos veces con distinto nivel de detalle: como lista de seis pasos con un ejemplo en la Sección 5 (pp. 344–345) y como pseudocódigo en la Sección 6 (pp. 345–346). Las dos versiones no dicen lo mismo. La restricción a los canales verde y azul aparece en el pseudocódigo pero no en la lista de pasos, donde recién aparece en el último párrafo de la Sección 5, después del ejemplo (p. 345). El lector debe reconciliar las dos versiones por su cuenta.

Faltan partes que este tipo de artículo suele tener. No hay una sección de trabajos relacionados, aunque el cuadro 2 de la página 347 compara contra tres trabajos ([9], [10] y [11]) que en el texto sólo se mencionan de pasada, para decir que las imágenes de prueba son “similar to previously use” en ellos (p. 346). No hay algoritmo de extracción para el método propuesto: la Sección 6 se titula “The proposed bit inversion algorithm” y contiene únicamente la inserción, mientras que la recuperación se cuenta en un párrafo en prosa de la página 345 (“So in de-steganography, ...”). Y no hay una sección de limitaciones.

La parte experimental tiene problemas propios. La Sección 7 anuncia que “we use three true RGB colour images (Lena.jpg , Baboon.jpg , Peppers.jpg)” (p. 346), pero la figura 2 de la misma página muestra seis imágenes y el cuadro 1 (p. 347) informa seis valores de PSNR. Los histogramas de las figuras 3 a 5 (pp. 346–347) no tienen títulos de ejes ni indican de qué imagen o canal son, y la comparación que se hace de ellos es puramente visual (“almost similar”, p. 347). Las afirmaciones centrales no tienen experimento que las respalde. El resumen dice que “less numbers of pixels are modified compared with the standard LSB method” (p. 342), pero ninguna tabla cuenta píxeles modificados ni informa el PSNR de LSB común sobre las mismas imágenes; el cuadro 1 sólo da el PSNR del método propuesto. La Sección 2 afirma que “Steg-analysis is performed on the standard LSB stego-image” (p. 343) y la conclusión habla de “undetectability” (p. 347), pero en ningún lugar del documento hay un análisis de detectabilidad ni una medición de capacidad. La conclusión repite casi textualmente los dos “niveles de seguridad” del resumen y cierra sin mencionar ninguna limitación.

La lista de referencias (p. 348) también es descuidada. El encabezado dice “REFERENCES”, la entrada [7] (“A survey of steganography technique”) no tiene medio de publicación ni año, la entrada [12] está escrita en minúsculas y las entradas mezclan formatos. La entrada [8] se usa para dos afirmaciones que no tienen relación entre sí, la cifra de los conos del ojo (p. 345) y la definición del PSNR (p. 347). Además, [8] es un trabajo de Rawat y Bhandari sobre esteganografía, no un trabajo de Hecht, que es el nombre con que el texto presenta la fuente de la primera afirmación.

En conjunto, consideramos que la organización no es la de un artículo terminado. El esquema anunciado falta, lo repetido ocupa lugar que hace falta para lo que falta (extracción, trabajos relacionados, análisis de detección) y las conclusiones no siguen de los experimentos presentados.

Descripción del algoritmo

Lo que sigue separa lo que se puede reconstruir del documento de lo que queda sin definir.

Lo que se puede reconstruir. La propiedad que hace decodificable al método es que LSB reemplaza sólo el bit 0 de cada byte y por lo tanto nunca toca los bits 1 y 2. El receptor puede entonces recalcular el patrón de cada byte del archivo estego y obtener los mismos grupos que tenía el emisor. El documento usa esa propiedad sin enunciarla: en la página 345 dice que en la extracción se debe “classify the stego-image according to the four patterns”, pero nunca explica por qué esa clasificación coincide con la del portador. Es también la razón por la que el paso 3 de la lista (recalcular los patrones sobre la imagen estego, p. 344) es redundante: los patrones de la imagen estego son los del portador. La regla de inversión (paso 5, p. 344) tiene una garantía que el documento tampoco enuncia. Si en un patrón hay n bytes que llevan mensaje y c cambiaron, después de invertir quedan $\min(c, n - c)$ cambios, o sea a lo sumo la mitad. Esa es la única justificación matemática de que el método reduzca modificaciones, y el documento la reemplaza por el ejemplo de cuatro valores de la página 345.

Lo que queda sin definir. El lugar, el tamaño y el formato del registro de patrones (“specific location”, pp. 344 y 346), y con eso su costo en capacidad, que el documento no cuenta. Cómo sabe el receptor cuánto mensaje leer, porque no hay campo de longitud. El orden de los bits del mensaje dentro de cada byte y el orden en que se recorren los bytes. Si un “pixel” es un píxel o un byte de color: el documento usa la palabra para las dos cosas, y en la página 345 dice “Three pixels that have ‘10’ pattern (A, B, D)” para hablar de tres valores de ocho bits. El orden de los dígitos del patrón: el ejemplo de la página 345 es coherente con que el bit 2 sea el dígito alto, pero no se dice. La regla de desempate cuando cambiaron tantos bytes como no cambiaron: el texto sólo dice “greater than” (paso 5, p. 344). Y qué se cuenta: todos los bytes del patrón, o sólo los que reciben un bit del mensaje. Esa última decisión cambia el resultado y el documento no la toma.

El paso 4b del ejemplo (p. 345) afirma que para el patrón ‘01’, con un solo valor (C), “we cannot check how many pixels were changed and how many were not, because there is only one pixel, comparison cannot occur”. La comparación sí puede hacerse: C no cambió (0 cambiados y 1 sin cambio), y la regla del paso 5 dice simplemente que no se invierte. Por otra parte, el ejemplo aplica el método a cuatro valores sueltos, sin indicar a qué canal pertenece cada uno, así que no ilustra la omisión del canal rojo que el método declara.

El pseudocódigo de la Sección 6 (pp. 345–346) tiene varios problemas, que se ven mejor en la imagen de la página que en el texto extraído. Primero, el bucle exterior recorre caracteres (“Get char from msg”) y el interior “For each 2 bits”, pero la condición “If msg bit = 1” nunca dice cuál de los bits es. Segundo, según la sangría de la página publicada las tres líneas “Replace the value in the pic”, “stego value = value” e “Identify the pattern of 2nd and 3rd bits of the stego value” quedan dentro de la rama Else, de modo que la rama del “1” no escribiría el valor en la imagen; consideramos que es un error de sangría y no una intención, pero el documento no permite decidirlo. Tercero, el segundo bucle usa “if number of changed pixels in any pattern of pic was more than pixels that not changed in alternative pattern in the stego”: “alternative pattern” no está definido y “any pattern” no aclara si la condición se evalúa por patrón o para alguno. Cuarto, dentro de esa condición el código recorre “each colour from green and blue” de cada píxel e invierte el bit sin volver a preguntar si el byte pertenece al patrón que superó la comparación; tal como está escrito, invertiría todos los bytes verdes y azules. Quinto, el registro (“Store the status of pattern that inversed his pixels as a map in specific location”) queda, por la misma sangría, dentro de los bucles más internos, es decir que se escribiría muchas veces. Además, la terminología cambia de una sección a otra: se usan “inverse”, “inversed” e “inversing” con el sentido de “invert” (pp. 344–346).

Resumen y declaraciones que no coinciden con el método. El resumen dice que el método “starts with the last LSBs of both green and blue colour planes that will be replaced by the first and the second most significant bits (MSB) of the secret image” (p. 342). Ninguna de las secciones 5 o 6 tiene un paso que use los bits más significativos de una imagen secreta: lo que se oculta es un mensaje de texto, bit a bit, en los bits menos significativos. Esa oración, a nuestro juicio, corresponde a otra versión del trabajo.

Argumentos de seguridad. El documento sostiene que usar sólo verde y azul agrega un “nivel de seguridad” porque “the red colour will act as noise data” (pp. 342, 345 y 347). Consideramos que es seguridad por oscuridad: quien conozca el método sabe que el rojo no lleva datos, y la protección se basa en mantener secreto el algoritmo, contrariamente al principio de Kerckhoffs [8]. La inversión de bits no agrega un secreto: el emisor y el receptor no comparten ninguna clave. La cita sobre los conos del ojo (“According to a research that was conducted by Hecht [8], 65% of all cones of human eyes are sensitive to red, 33% ... green, and only near about 2% ... blue”, p. 345) es un argumento perceptivo, sobre qué canal conviene no modificar, pero el párrafo lo presenta como una mejora de “the security”. Además, a nuestro juicio, la cifra describe la proporción de conos de cada tipo y no la sensibilidad de la imagen al ruido, y la atribución a Hecht no coincide con la entrada [8] (p. 348), como se dijo antes.

Capacidad. La Sección 2 dice que la capacidad de LSB estándar es baja porque “only one bit per pixel is used to hide the data” (p. 343), pero unas líneas antes el propio documento explica que cada píxel tiene tres componentes de color (pp. 342–343), de modo que LSB estándar oculta tres bits por píxel, uno por byte de color. Sólo tiene sentido si “pixel” significa “byte”, que es justamente la ambigüedad de vocabulario ya señalada. La misma sección afirma que el trabajo toma en cuenta “to increase the capacity” y que “any security level improvement must not affect both capacity and the quality of stego image” (p. 343). Sin embargo, si sólo se usan dos de los tres canales, la capacidad baja a dos tercios. Con la implementación de este trabajo, el mismo portador `lado.bmp` admite 115 200 bytes con LSB1 y 76 799 bytes con LSBI, porque además del canal rojo hay que descontar el registro de cuatro banderas. El documento nunca mide ni discute esa pérdida.

Robustez. La Sección 2 define la robustez como “the difficulty of any unauthorized party to retrieve the secret data” (p. 343), y la palabra vuelve como palabra clave (p. 342). En la literatura de esteganografía [6] lo que se llama robustez es la supervivencia del mensaje ante modificaciones de la imagen estego (compresión, recorte, ruido) y lo que se define como el documento es más bien seguridad o indetectabilidad. Consideramos que la elección del término impide entender qué propiedad dice mejorar.

Evaluación con PSNR. El cuadro 2 (p. 347) compara el PSNR del método contra los de tres trabajos ([9] a [11]) sin decir el tamaño de las imágenes, la cantidad de datos ocultos en cada caso ni con qué mensaje se obtuvieron esas cifras. El único mensaje del documento tiene 226 caracteres (p. 346). Como el PSNR depende sobre todo de cuántos bytes se modifican, comparar valores obtenidos con cargas distintas no mide la calidad del método. Nuestras mediciones lo muestran: al ocultar el mismo mensaje del documento en `lado.bmp`, LSB1, LSB4 y LSBI dan 77,56, 60,91 y 78,56 dB, mientras que con la imagen PNG de la cátedra el mismo LSBI baja a 55,45 dB por la sola diferencia de la carga (véase la Cuestión II). Por último, los portadores se nombran con extensión `.jpg`, y el documento no aclara en qué formato se guardaron las imágenes estego; si se recomprimieran con pérdida, el mensaje oculto en los bits menos significativos no sobreviviría.

Lo que el documento deja abierto y lo que decidieron los vectores. Al reproducir byte a byte el archivo de la cátedra `Ejemplo/ladoLSBI.bmp` con esta implementación aparecieron todas las ambigüedades anteriores, una por una. El cuadro 1 resume, para cada aspecto, qué dice el documento, qué agrega la consigna y qué resolvieron los vectores de la cátedra (los tres archivos

LSBI de Ejemplo/). En seis de los diez aspectos un vector de la cátedra descarta las lecturas alternativas; en los otros cuatro (el orden de los dígitos del patrón, el desempate, la capacidad y el relleno de filas) los vectores no permiten decidir y la implementación adopta la lectura literal del documento y de la consigna. Los detalles de cada prueba están en la Cuestión IX.

Notación: claridad, errores y contradicciones

La notación no es clara. Los errores del cuadro 2 son verificables en el texto o en la imagen de la página citada; algunos son tipográficos y otros afectan la comprensión del método. El más grave para el lector que quiere implementar el método es la lista de patrones del paso 1 (p. 344), que dice “either 00, 10, 10, or 11”: el patrón 10 aparece dos veces y falta el 01. El pseudocódigo de la página 345 sí lista los cuatro patrones correctamente (“00, 01, 10, or 11”), así que se trata de un error de tipeo, pero es uno que en un texto sobre patrones de dos bits obliga a adivinar. En el ejemplo de la página 345, el valor D aparece con siete bits (1010110) en la tabla del paso 2, mientras que el paso 1 del mismo ejemplo y las líneas siguientes lo escriben con ocho (10101101). Como el ejemplo se basa en los bits 1 y 2, la diferencia importa: leído tal cual, el valor de siete bits tendría los bits 2 y 1 en 11, y no en 10, que es el patrón que el ejemplo le asigna a D.

En cuanto a la claridad general, consideramos que el texto es difícil de seguir por tres razones. Usa la misma palabra (“pixel”) para tres cosas distintas: un píxel, un byte de color y una posición de arreglo. Alterna entre “LSB”, “last bit” y “last significant bit” para el mismo objeto. Y presenta los ejemplos con valores sueltos (A, B, C, D) sin decir de qué imagen ni de qué canal provienen, de modo que el lector no puede reproducirlos en una imagen real. Además hay errores tipográficos que en un texto técnico dificultan la lectura: “lest significant bit” (p. 345, dos veces en el pseudocódigo), “massage bits” (p. 345), “two has changed” (p. 345), “In In steganography” (p. 347), “We would like to emphasis” (p. 347), “this this proposed method” (p. 348) y “REFERENCES” (p. 348). Ninguno de ellos cambia el algoritmo, pero en conjunto refuerzan la impresión de que el texto no tuvo una revisión final.

Conclusión

El documento tiene una idea correcta y aprovechable: agrupar los bytes por los dos bits que LSB no modifica y decidir por grupo si conviene invertir el bit menos significativo, de modo que nunca cambie más de la mitad de los bytes de cada grupo. Consideramos que la descripción no alcanza para implementarla sin decisiones propias, porque el lugar y el formato del registro, el orden de bits, el desempate y el alcance del conteo quedan sin definir, y la consigna sólo resuelve algunos de esos puntos. Además contiene errores de notación verificables, entre ellos la lista de patrones con un patrón repetido, un valor del ejemplo con siete bits, los ejes de la figura 1 intercambiados y la cuenta de bits del mensaje, y sus afirmaciones de seguridad e indetectabilidad no están respaldadas por ningún experimento del propio documento. La mejora que sí se puede medir, la reducción de bytes modificados, se analiza con datos propios en la Cuestión VII.

Cuestión II: Comparación de LSB1, LSB4 y LSBI

Enunciado. *Esteganografiar un mismo archivo en un .bmp con cada uno de los tres algoritmos, y comparar los resultados obtenidos. Hacer un cuadro comparativo de los tres algoritmos estableciendo ventajas y desventajas.*

Aspecto	Paper	Consigna	Vectores de la cátedra
Lugar y orden de las banderas	“specific location” (pp. 344, 346), sin definir	Cuatro banderas en los primeros 4 bytes, LSB1 normal, tres canales, antes del mensaje; orden 00, 01, 10, 11	Decidido: el orden inverso da un tamaño absurdo y encuadre inválido; el orden de la consigna reproduce el archivo
Canales que llevan mensaje	Sólo verde y azul (pp. 342, 345); el ejemplo de la p. 345 no distingue canales	Orden físico B, G, R; no dice que el rojo se omite	Decidido: usar el rojo o contar su posición desde el byte 4 no reproduce el archivo; el rojo se omite contando desde el inicio del bloque
Inicio del mensaje y orden de bits	Sin definir (“index j_i ”, p. 344)	Formato tamaño, datos, extensión; las banderas suman 4 bytes	Decidido: el mensaje empieza en el byte 4 del bloque y va del bit más significativo al menos significativo
Orden de los dígitos del patrón	“2nd last and 3rd last bit” (p. 344); el ejemplo de la p. 345 sugiere bit 2 como dígito alto	Lista los patrones 00, 01, 10, 11 sin definir los dígitos	Sin decidir: en los tres vectores las banderas de 01 y 10 son iguales, y cambiar el orden da los mismos bytes
Qué bytes se cuentan	“pixels” cambiados y no cambiados (pp. 344–345), sin precisar	Nada	Decidido: sólo los bytes verdes y azules que reciben un bit del mensaje; contar el rojo no reproduce el archivo
Desempate	Sólo “greater than” (p. 344)	Nada	Sin decidir: ningún vector tiene empates; se invierte sólo si cambiaron más que los que no
Alcance de la inversión	“Inverse the LSB bits” (p. 344)	Nada	Decidido: sólo el bit 0 de los bytes con mensaje se combina con la bandera; el resto queda intacto
Capacidad	No la calcula ni la discute	4 bits más; error con la capacidad máxima	Sin decidir: los vectores sólo prueban que 44 895 bytes entran; la fórmula de este trabajo da 76 799 bytes para <code>lado.bmp</code>
Relleno de filas	Trata la imagen como arreglo plano	Usar el bloque de píxeles más los bytes de relleno	Sin decidir: <code>lado.bmp</code> no tiene relleno; se usa el índice plano
Polaridad de las banderas	“status of the patterns that inverse its pixel” (p. 344), implícita	1 indica que el patrón tuvo cambio; las banderas no se invierten	Decidido: la polaridad contraria da encuadre inválido; las banderas se guardan tal cual

Cuadro 1: Lo que el documento deja abierto en LSBI y lo que resolvieron los vectores de la cátedra al reproducir `Ejemplo/ladoLSBI.bmp`

Página	Texto del documento	Problema	Corrección
p. 344	“either 00, 10, 10, or 11” (paso 1)	El patrón 10 se repite y falta 01	00, 01, 10, 11 (como en la p. 345)
p. 345	D = 1010110 en la tabla del paso 2	Tiene 7 bits; el paso 1 y las líneas siguientes usan 10101101	10101101
p. 343	Fig. 1: ejes “BLUE (0,1,0)” y “GREEN (0,0,1)”	Verde y azul están intercambiados; rojo (1,0,0) es correcto	GREEN (0,1,0) y BLUE (0,0,1)
p. 343	“(28)3 = 16,777,216”	Los exponentes se perdieron; se lee 28 por 3	$(2^8)^3 = 2^{24} = 16\,777\,216$
p. 344	Sección 4: “For i = 1 to Length(c), do $S_j \leftarrow C_j$ ”; “ $S_{j_i} \leftarrow \text{LSB}(C_{j_i}) = m_i$ ”	El índice del bucle es i y el cuerpo usa j ; “c” y “C” se mezclan; la asignación lleva una igualdad adentro	$S_j \leftarrow C_j$ para todo j ; $\text{LSB}(S_{j_i}) \leftarrow m_i$
p. 344	Extracción: “Input -: Secret image s”; “Compute index j_i where to store the i th message bit”; $m_{j_i} \leftarrow \text{LSB}(C_{j_i})$	La entrada es la imagen estego, no una “secret image”; se lee del portador C cuando el texto siguiente dice que no hace falta; el índice es de lectura y el resultado va a m_{j_i}	Entrada S ; $m_i \leftarrow \text{LSB}(S_{j_i})$
p. 346	“The following text includes 226 characters or 2881 bits”	226 caracteres por 8 bits son 1808 bits; 2881 no es múltiplo de 8 (ni de 7)	226 caracteres, 1808 bits
p. 347	“cover image C of size $M \times M$ and the stego-image S of size $N \times N$ ”; MSE con $\alpha_{i,j}$ y $\beta_{i,j}$; MAX sin definir	El MSE resta píxel a píxel y suma sobre M por N , lo que exige el mismo tamaño; el texto usa (x, y) y la fórmula (i, j) ; C , S pasan a α , β ; MAX no se define	Ambas imágenes de $M \times N$; una sola notación; MAX = 255
pp. 346–347	“three true RGB colour images” (Sección 7), pero la Fig. 2 muestra seis y el cuadro 1 informa seis PSNR	Contradicción entre el texto y sus figuras y tablas	Decir seis imágenes
pp. 342, 346, 347	Nombres: “Babbon.jpg” (resumen y rótulo de la Fig. 2) frente a “Baboon.jpg”; “Pepper.jpg” frente a “Peppers.jpg”; “trees.jpg” frente a “tree . jpg”; “Tiffany.jpg” frente a “tifanny. Jpg”	Los nombres de las imágenes no coinciden entre resumen, figura y tablas	Un nombre por imagen
pp. 345–347	“pixel” para un byte de color; “secret image” para un mensaje de texto	Confunde la unidad de análisis y la naturaleza del mensaje (“Three pixels that have ‘10’ pattern”, p. 345; “the bits of the secret image”, p. 347)	Byte o componente de color; mensaje
pp. 344–346	“inverse”, “inversed”, “inversing”	Se usa “inverse” con el sentido de “invert”	“invert”, “inverted”

Cuadro 2: Errores y contradicciones de notación en el documento

Metodología

El portador es `Ejemplo/lado.bmp`, un BMP de 640×480 píxeles y 24 bits por píxel, con 921 600 bytes de datos de imagen y sin relleno al final de cada fila. Para que la comparación no dependa de un único mensaje se ocultaron tres cargas con cada método: la imagen PNG que la cátedra escondió en sus propios ejemplos (44 886 bytes, un archivo ya comprimido), el mensaje de prueba del propio paper [1] (226 bytes de texto en inglés, sin retorno de carro final) y 40 000 bytes pseudoaleatorios, generados de forma determinista con SHA-256, que hacen de reemplazo de un mensaje cifrado (un buen cifrado produce bytes indistinguibles del azar). La carga principal es el PNG, y con él se escribieron los comandos:

```
./stegobmp -embed -in catedra.png -p Ejemplo/lado.bmp -out png-LSB1.bmp -steg LSB1
./stegobmp -embed -in catedra.png -p Ejemplo/lado.bmp -out png-LSB4.bmp -steg LSB4
./stegobmp -embed -in catedra.png -p Ejemplo/lado.bmp -out png-LSBI.bmp -steg LSBI
```

Cada resultado se extrajo de nuevo con `-extract` y se comparó byte a byte con la carga original, y los tres archivos con el PNG se compararon con los ejemplos de la cátedra (`ladoLSB1.bmp`, `ladoLSB4.bmp` y `ladoLSBI.bmp`): son idénticos byte a byte, de modo que las cifras que siguen describen también la implementación de referencia. La capacidad máxima de cada método se leyó del mensaje de error que imprime `stegobmp` al intentar ocultar un archivo demasiado grande. La distorsión se midió con `tools/bmpdiff.py` (bytes y bits modificados, cambios por canal, error cuadrático medio MSE y relación señal-ruido PSNR sobre los bytes de color) y las huellas estadísticas con `tools/lsbstats.py` (proporción de bytes con el bit menos significativo en uno y estadístico χ^2 de pares de valores, por canal, definido como en Westfeld y Pfitzmann [4]). Todo se regenera con `make report`, sin intervención manual.

Resultados

El cuadro 3 resume el caso del PNG. El cuadro 4 repite las mediciones esenciales para las tres cargas y el cuadro 5 muestra las huellas estadísticas por canal del portador original y de los tres archivos con el PNG.

Métrica	LSB1	LSB4	LSBI
Capacidad máxima (bytes)	115 200	460 800	76 799
Bytes ocultos	44 895	44 895	44 895
Uso de la capacidad (%)	38,97	9,74	58,46
Bytes del portador usados	359 160	89 790	359 164
Bytes modificados	190 182	85 370	170 852
Bits modificados	190 182	205 965	170 852
Tasa de cambio por byte usado (%)	52,95	95,08	47,57
Bytes modificados sobre el total (%)	20,64	9,26	18,54
Cambio máximo por byte	1	15	1
Bytes rojos modificados	63 341	28 443	0
Último offset modificado	359 213	89 843	538 794
MSE	0,2064	8,7631	0,1854
PSNR (dB)	54,98	38,70	55,45
Idéntico al archivo de la cátedra	sí	sí	sí
Extracción correcta	sí	sí	sí

Cuadro 3: Resultados medidos al ocultar la misma imagen PNG en `lado.bmp` con cada método

Carga	Método	Ocultos	Uso (%)	Modificados	Tasa (%)	PSNR (dB)
Imagen PNG	LSB1	44 895	38,97	190 182	52,95	54,98
	LSB4	44 895	9,74	85 370	95,08	38,70
	LSBI	44 895	58,46	170 852	47,57	55,45
Mensaje del paper	LSB1	235	0,20	1 051	55,90	77,56
	LSB4	235	0,05	454	96,60	60,91
	LSBI	235	0,31	835	44,32	78,56
Datos aleatorios	LSB1	40 009	34,73	159 834	49,94	55,74
	LSB4	40 009	8,68	74 983	93,71	40,54
	LSBI	40 009	52,10	159 773	49,92	55,74

Cuadro 4: Las tres cargas ocultas con cada método. Ocultos: bytes ocultados (con tamaño y extensión); Uso: porcentaje de la capacidad; Modificados: bytes del portador que cambiaron; Tasa: porcentaje de los bytes usados que cambiaron. Un texto corto, una imagen comprimida y datos aleatorios (cifrados) se comportan de manera distinta

Archivo	Proporción de LSB igual a 1			χ^2 de pares de valores		
	B	G	R	B	G	R
Portador original	0,5475	0,5442	0,5488	8475,17	7592,63	8606,07
PNG con LSB1	0,4582	0,4571	0,4577	2125,38	2144,94	2028,76
PNG con LSB4	0,5137	0,5108	0,5147	5232,30	4372,81	5226,22
PNG con LSBI	0,5626	0,5608	0,5488	3520,98	3345,95	8606,07

Cuadro 5: Estadísticas del bit menos significativo por canal, medidas sobre todo el bloque de píxeles del portador y de los tres archivos con el PNG



(a) Portador original



(b) Con el PNG oculto usando LSB4

Figura 1: El portador y el resultado de LSB4, el método más agresivo por byte: no hay diferencia visible a simple vista

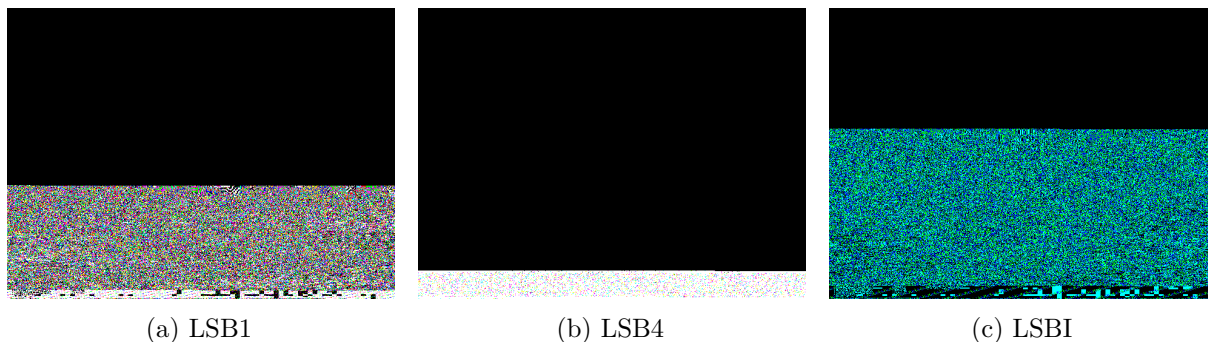


Figura 2: Mapas de cambios respecto del portador con el PNG oculto. Cada canal se enciende (rojo, verde o azul) en los píxeles donde ese byte cambió; el resto queda en negro. Las filas se muestran de arriba hacia abajo, pero el BMP guarda la fila inferior primero, así que los cambios empiezan en la parte de abajo de la imagen

Análisis

Capacidad. LSB4 ofrece exactamente cuatro veces la capacidad de LSB1 (460 800 contra 115 200 bytes), porque aloja cuatro bits por byte de portador en lugar de uno. LSBI queda por debajo de LSB1 (76 799 bytes): sus bits de mensaje solo van a los bytes verde y azul, es decir, dos tercios del bloque, y además consume cuatro bytes de portador para las banderas de los patrones invertidos. El mismo PNG ocupa el 38,97 % de la capacidad de LSB1, el 9,74 % de la de LSB4 y el 58,46 % de la de LSBI: el método con el mayor margen es LSB4, y LSBI es el primero que se queda sin lugar si el archivo crece.

Distorsión por byte. LSB1 y LSBI modifican como mucho el último bit, de modo que cada byte cambia a lo sumo en 1 nivel; LSB4 reemplaza los cuatro bits bajos y un byte puede cambiar hasta 15 niveles. Por eso, aunque LSB4 toca menos bytes (85 370, contra 190 182 de LSB1 y 170 852 de LSBI), su MSE es mucho mayor (8,7631, contra 0,2064 y 0,1854) y su PSNR más bajo (38,70 dB, contra 54,98 dB y 55,45 dB). Aun así, con esta carga LSB4 queda en un valor de PSNR que suele considerarse imperceptible, y la figura 1 lo confirma a simple vista.

Cuántos bytes cambian y dónde. LSB4 concentra sus cambios en los primeros 89 790 bytes del bloque (el último byte modificado es el de posición 89 843), una franja angosta en la parte inferior de la figura 2. LSB1 necesita ocho bytes por byte de mensaje y los reparte en una zona cuatro veces mayor (hasta la posición 359 213), con los tres canales por igual. LSBI llega más lejos (538 794) porque salta cada tercer byte, y el mapa muestra su rasgo característico: el canal rojo no se enciende nunca (0 bytes rojos modificados). En cambio, LSB1 modificó 63 341 bytes rojos y LSB4 28 443.

Tasa de cambio. Sobre los bytes que efectivamente llevan mensaje, LSB1 cambió el 52,95 %, algo más que la mitad que resultaría de combinar bits equiprobables con bits equiprobables; la diferencia refleja que ni el PNG ni los bits bajos del portador lo son exactamente (la proporción de unos del portador es 0,5475 en el canal azul). LSB4 cambió el 95,08 %, como corresponde a reemplazar cuatro bits: casi ningún byte coincide por casualidad, ya que con bits equiprobables solo uno de cada dieciséis lo haría. LSBI bajó al 47,57 % gracias a la inversión de patrones, una mejora real pero pequeña, cuyo mecanismo se explica en la Cuestión VII.

Huellas estadísticas. El estadístico χ^2 del portador vale, por canal, 8475,17, 7592,63 y 8606,07; en los tres archivos cae en los canales donde se incrustó (por ejemplo, el canal azul baja a 2125,38 con LSB1, 5232,30 con LSB4 y 3520,98 con LSBI), porque un mensaje casi aleatorio iguala los conteos de cada par de valores. LSBI deja el canal rojo con 8606,07, el mismo valor del portador. Esa asimetría entre canales es coherente con el método, pero también es una pista para quien

analice el archivo. Como LSB1 y LSB4 recorren el bloque desde el primer byte y en orden, dejan una zona inicial con estadística de azar y el resto intacto, que es justamente lo que aprovecha el ataque de Westfeld y Pfitzmann [4].

Otras cargas. Con el mensaje del paper cambian muy pocos bytes (1 051 con LSB1, 454 con LSB4 y 835 con LSBI) y los tres métodos alcanzan un PSNR muy alto (77,56, 60,91 y 78,56 dB): en ese régimen el PSNR mide sobre todo el tamaño del mensaje, no la calidad del método. Con datos aleatorios, que representan un mensaje cifrado, LSBI casi deja de ganar: cambia el 49,92 % de los bytes usados contra el 49,94 % de LSB1, y la inversión reduce los cambios de los bytes con mensaje en un 0,16 %, valor del mismo orden que lo esperable para bits equiprobables (unos 439 cambios, el 0,27 %). La ventaja de LSBI existe cuando el mensaje tiene estructura (el PNG reduce un 9,27 %, el texto del paper un 20,31 %), y desaparece justamente en el caso que más importa para un programa que también cifra. Esto se reporta a propósito junto al resultado favorable del PNG y no debe leerse como una mejora general.

Cuadro comparativo

El cuadro 6 resume las ventajas y desventajas de cada método, con las cifras medidas en este informe.

Método	Ventajas	Desventajas
LSB1	Implementación muy simple y sin metadatos. Distorsión mínima por byte (cambio máximo de 1 nivel; PSNR 54,98 dB con el PNG). Usa los tres canales, con capacidad 115 200 bytes. No depende de la estadística del mensaje.	Capacidad cuatro veces menor que la de LSB4. Modifica los tres canales y desplaza sus estadísticas, y al recorrer el portador en orden fijo desde el byte cero es detectable con el χ^2 de pares de valores. Cambia cerca de la mitad de los bytes que usa (52,95 %).
LSB4	Mayor capacidad (460 800 bytes): el PNG usa solo el 9,74 % y sus cambios se concentran en los primeros 89 790 bytes, el 9,26 % del portador. Implementación igual de simple que la de LSB1.	Cambia casi todos los bytes que usa (95,08 %) y hasta 15 niveles, con MSE 8,7631 y PSNR 38,70 dB, el peor de los tres. La franja de cambios es compacta y fácil de localizar, y cambia la estadística de los tres canales.
LSBI	Menor tasa de cambio (47,57 %) y menor MSE (0,1854) que LSB1 con archivos estructurados. Deja el canal rojo intacto (0 bytes modificados). Los cambios quedan repartidos en una zona más extensa del portador, con menor densidad local.	Capacidad menor (76 799 bytes) y cuatro bytes de banderas. Algoritmo más complejo (conteo por patrón y dos pasadas). La ganancia depende del mensaje: con datos aleatorios o cifrados es casi nula (0,16 %). El canal rojo intacto revela una asimetría entre canales.

Cuadro 6: Ventajas y desventajas de LSB1, LSB4 y LSBI

Los tres métodos comparten las mismas limitaciones de fondo: cualquier recompresión con pérdida, cambio de tamaño o conversión de formato destruye el mensaje, y ninguno protege el contenido por sí mismo; por eso el programa ofrece cifrado previo. Ninguno resiste un análisis estadístico dirigido: el cifrado previo protege el contenido del mensaje, pero no su existencia.

Conclusión de la comparación

LSB4 conviene cuando importa la capacidad y el portador tolera una distorsión moderada: es el método con mayor margen y el que menos superficie del portador toca, pero es el que más cambia cada byte. LSB1 es el punto medio: capacidad suficiente para archivos medianos, distorsión mínima por byte y ninguna complejidad extra. LSBI se justifica cuando el mensaje

tiene estructura y se quiere reducir aún más la cantidad de bytes modificados sin tocar el canal rojo, a cambio de un tercio de capacidad y de un algoritmo más delicado; si el mensaje ya está cifrado, su ventaja práctica es marginal y LSB1 resulta más simple con resultados casi idénticos.

Cuestión III: Procedimiento de estegoanálisis

Enunciado. *Explicar detalladamente el procedimiento realizado para descubrir qué se había ocultado en cada archivo y de qué modo. Indicar qué se encontró en cada archivo.*

Pendiente: esta cuestión requiere los archivos de estegoanálisis de la cátedra, que todavía no fueron entregados.

Cuestión IV: Mensajes ocultos dentro de mensajes ocultos

Enunciado. *Algunos mensajes ocultos tenían, a su vez, otros mensajes ocultos. Indica cuál era ese mensaje y cómo se había ocultado.*

Pendiente: esta cuestión requiere los archivos de estegoanálisis de la cátedra, que todavía no fueron entregados.

Cuestión V: El fragmento de película

Enunciado. *Uno de los archivos ocultos era una porción de un video de una película, donde se ve ejemplificado una manera de ocultar información ¿qué se ocultaba y sobre qué portador?*

Pendiente: esta cuestión requiere los archivos de estegoanálisis de la cátedra, que todavía no fueron entregados.

Cuestión VI: El método de ocultamiento desconocido

Enunciado. *¿De qué se trató el método de esteganografiado que no era LSB1 ni LSB4 ni LSBI? ¿Es un método eficaz? ¿Por qué?*

Pendiente: esta cuestión requiere los archivos de estegoanálisis de la cátedra, que todavía no fueron entregados.

Cuestión VII: Por qué LSBI mejora a LSB

Enunciado. *¿por qué la propuesta del documento de Majeed y Sulaiman es realmente una mejora respecto de LSB común?*

El mecanismo

LSB común reemplaza el bit menos significativo de cada byte que usa por un bit del mensaje. Si los bits del mensaje y los bits bajos del portador no guardan relación, cada byte usado cambia con probabilidad un medio, de modo que se modifica alrededor de la mitad de los bytes que llevan mensaje. Majeed y Sulaiman [1] aprovechan una propiedad que el propio documento no enuncia: LSB sólo toca el bit 0 de cada byte, así que los bits 1 y 2 son idénticos en el portador y en el archivo estego. Con esos dos bits se arman cuatro grupos (patrones 00, 01, 10 y 11) que el emisor y el receptor calculan de la misma manera, sin necesidad de intercambiar nada.

Para cada grupo, el emisor cuenta cuántos de sus bytes con mensaje cambiarían con LSB común (sea c) y cuántos quedarían igual ($n - c$, con n el total del grupo). Si cambiarían más de los que no ($c > n - c$), en ese grupo escribe el complemento de los bits del mensaje, de modo que ahora cambian justo los $n - c$ bytes que antes no cambiaban, y deja la bandera del grupo en uno. El receptor lee las cuatro banderas, recalcula el patrón de cada byte y aplica la bandera de su grupo para recuperar el bit original. Por grupo, la cantidad final de bytes modificados es

$$\min(c, n - c) \leq \min(c, \frac{n}{2}),$$

donde un empate no se invierte y deja $c = n - c$. Sumando los cuatro grupos, LSBI nunca modifica más bytes con mensaje que LSB común sobre las mismas posiciones, y a lo sumo modifica la mitad de los bytes de cada grupo. El costo es de cuatro bytes de banderas, escritos con LSB1, que pueden agregar como mucho cuatro cambios. La ganancia en cada grupo es $|c - n/2|$, es decir, cuánto se aparta el conteo del punto medio: cuanto más desbalanceado sea el grupo, más se gana.

Evidencia medida

El cuadro 7 compara, sobre los mismos bytes, los cambios sin inversión (lo que haría LSB común en esas posiciones) con los cambios con inversión, para cinco cargas: el PNG de la cátedra, el mensaje de prueba del paper, datos aleatorios y los dos vectores cifrados de la cátedra. La última columna cuenta los cambios que producen las banderas mismas.

Caso	Bytes con mensaje	Sin inversión	Con inversión	Reducción (%)	Banderas	Cambios en banderas
PNG (sin encriptación)	359 160	188 308	170 852	9,27	1111	0
Mensaje del paper	1 880	1 044	832	20,31	0001	3
Datos aleatorios	320 072	160 032	159 771	0,16	0101	2
PNG aes256-ofb (cátedra)	359 192	179 491	179 329	0,09	0001	3
PNG 3des-cfb (cátedra)	359 192	179 425	179 195	0,13	1000	3

Cuadro 7: Bytes con mensaje modificados sin y con inversión de patrones, para cinco cargas ocultas en `lado.bmp` con LSBI. La reducción se calcula sobre los bytes con mensaje; las banderas se cuentan aparte

Con el PNG, la inversión baja los cambios de 188 308 a 170 852, una reducción del 9,27 %, y las banderas no agregan ningún cambio. El cuadro 8 muestra de dónde sale esa ganancia.

Los cuatro grupos se invierten, pero en tres de ellos la diferencia entre cambiados y sin cambio es mínima, y casi toda la reducción sale del patrón 11, que es el más numeroso y el de mayor desequilibrio. La explicación es una propiedad del portador y no del método: en `lado.bmp` hay muchas zonas saturadas, cuyos bytes valen 255, tienen el patrón 11 y el bit menos significativo en uno; en ese grupo los bits bajos del portador son, por lo tanto, mayoritariamente unos (un conteo auxiliar sobre `lado.bmp` lo confirma), mientras que el PNG, un archivo comprimido, tiene menos unos que ceros; así, en ese grupo cambian más bytes de los que quedan. Es coherente

Patrón	Cambiados	Sin cambio	Invertido	Cambios finales
00	32 367	32 028	sí	32 028
01	32 650	32 263	sí	32 263
10	35 119	34 980	sí	34 980
11	88 172	71 581	sí	71 581
Total	188 308	170 852		170 852

Cuadro 8: Conteo por patrón al ocultar el PNG de la cátedra con LSBI: bytes que cambiarían, bytes que quedarían igual, si el grupo se invierte y cambios que quedan después

con la proporción global de unos del canal azul del portador (0,5475, mayor que un medio). En otras palabras, LSBI gana cuando el portador y el mensaje tienen estadísticas distintas dentro de algún patrón, y esa diferencia es propia del par portador-mensaje.

Los casos cifrados y el aleatorio confirman esto. Con datos aleatorios y con los dos vectores cifrados de la cátedra (AES-256 en OFB y 3DES en CFB) la reducción es de 0,16 %, 0,09 % y 0,13 %: mínima. Un mensaje cifrado se comporta como una secuencia de bits equiprobables e independientes del portador. En cada grupo el conteo c es entonces una variable binomial de media $n/2$, y la ganancia esperada es su desviación absoluta media, aproximadamente $\sqrt{n/(2\pi)}$ por grupo. Para la carga aleatoria eso suma unos 439 cambios, el 0,27 % de los bytes con mensaje. Lo medido queda por debajo pero del mismo orden, con la dispersión propia de una única muestra. La ganancia proviene del azar del muestreo y no de una regularidad que el método aproveche; además, con las banderas, la ganancia neta baja todavía más (hasta 3 cambios en banderas en los vectores cifrados).

La comparación con LSB1 sobre el mismo PNG (Cuestión II) va en el mismo sentido. LSB1 usó 359 160 bytes y cambió 190 182, una tasa del 52,95 %; LSBI usó 359 164 bytes (contando las banderas) y cambió 170 852, una tasa del 47,57 %. El PSNR pasa de 54,98 a 55,45 dB: una mejora de una fracción de decibel, coherente con una reducción de cambios de menos del diez por ciento. Con el mensaje corto del paper los cambios pasan de 1 051 a 835, y el PSNR de 77,56 a 78,56 dB, pero en ese régimen ambas cifras son muy altas y no dicen mucho sobre la calidad del método. La reducción relativa del texto del paper es la mayor de las cinco cargas (20,31 %), y es esperable: un texto ASCII tiene siempre en cero el bit más significativo de cada carácter y sus bits están lejos de ser equiprobables. Pero se calcula sobre 1 880 bytes solamente y con 3 cambios de banderas que hay que sumar aparte, así que la ganancia neta es algo menor.

¿Es realmente una mejora?

En sentido estricto y medible, sí: LSBI no modifica más bytes con mensaje que LSB común sobre las mismas posiciones (a lo sumo cuatro cambios más por las banderas), y cuando el mensaje tiene una estadística distinta de la del portador modifica menos, con un PSNR algo mayor. El precio y los límites de esa mejora son estos.

La capacidad baja. Como los bits de mensaje sólo van a los bytes verde y azul, y además hay que reservar las banderas, el mismo `lado.bmp` admite 76 799 bytes con LSBI contra 115 200 con LSB1. El documento dice tener en cuenta la capacidad (p. 343), pero no la mide.

La ganancia depende del mensaje. Justamente en el caso que más importa para este programa, un mensaje cifrado, la reducción es casi nula (cuadro 7), y el algoritmo es más complejo que LSB común: dos pasadas y un conteo por patrón.

No vuelve indetectable el mensaje. El estadístico χ^2 de pares de valores de Westfeld y Pfitzmann [4], medido en el cuadro 5, baja en el canal azul de 8475,17 a 3520,98 y en el verde de

7592,63 a 3345,95: LSBI sigue siendo un reemplazo de bits menos significativos, y un mensaje casi aleatorio iguala los conteos de cada par de valores haya inversión o no. El análisis RS de Fridrich, Goljan y Du [5] apunta a la misma asimetría del reemplazo de LSB; no lo aplicamos, y no pretendemos que LSBI lo evada ni que lo detecte peor. El canal rojo, además, queda con el mismo estadístico del portador (8606,07), así que la diferencia entre canales es una marca por sí misma, visible en la figura 2.

La seguridad adicional es oscuridad. Que el rojo “actúe como ruido” sólo protege mientras el método sea secreto; quien lo conoce sabe qué canales mirar, contra el principio de Kerckhoffs [8]. El mensaje, además, empieza siempre en el mismo byte y se recorre en orden secuencial, con lo que la extracción es trivial para cualquiera que conozca el método. La inversión de patrones no agrega ninguna clave.

En conclusión, consideramos que LSBI es una mejora real pero estrecha. Reduce la cantidad de bytes modificados, y eleva algo el PSNR, cuando la estadística del mensaje difiere de la del portador dentro de algún patrón. A cambio pierde un tercio de la capacidad más cuatro bytes, y su ventaja se diluye cuando el mensaje se cifra, que es la práctica recomendable. No mejora la resistencia al estegoanálisis estadístico ni justifica, con lo medido, decir que el mensaje pasa inadvertido: los rastros estadísticos de los canales verde y azul, y la asimetría con el rojo, siguen presentes.

Cuestión VIII: Otras formas de guardar los patrones invertidos

***Enunciado.** En la implementación se optó por guardar los patrones invertidos antes del mensaje ¿de qué otra manera o en qué otro lugar podría guardarse el registro de los patrones invertidos?*

La elección actual

La consigna [2] (páginas 4 y 5) fija dónde se guarda el registro: cuatro bits, uno por patrón (00, 01, 10 y 11), escritos con LSB1 normal, sin invertir, en los cuatro primeros bytes del bloque de píxeles y usando los tres canales, antes de los bits del mensaje. Un uno significa que el patrón se invirtió. Esta implementación sigue esa regla, y los vectores de la cátedra la confirman (véase la Cuestión I y la Cuestión IX).

Tiene tres ventajas. La posición es fija y conocida, de modo que el receptor puede leer las banderas antes de decodificar nada. No depende del largo del mensaje ni de una clave. Y su costo es mínimo: cuatro bytes del portador. Tiene también desventajas. La posición es previsible para cualquiera que conozca el método. Una sola bandera dañada invierte un grupo entero de bytes y arruina el mensaje, porque no hay redundancia. Y, como se usan los tres canales, el tercero de los cuatro bytes es rojo, y es el único byte rojo que LSBI puede modificar (con la carga del paper cambió 1 byte rojo).

Alternativas

El cuadro 9 compara la elección actual con otras nueve. Las columnas son el costo en capacidad, la detectabilidad (qué tan fácil es que alguien note o localice el registro) y la compatibilidad y robustez (qué exige del receptor, si sobrevive a cambios de la imagen y si es compatible con los archivos de la cátedra). Las valoraciones son un análisis propio. Salvo la primera, ninguna alternativa es compatible con los archivos LSBI de la cátedra, porque éstos traen las banderas donde manda la consigna.

Alternativa	Dónde / cómo	Costo en capacidad	Detectabilidad	Compatibilidad y robustez
1. Elección actual (referencia)	Cuatro bits con LSB1 en los primeros 4 bytes del bloque, antes del mensaje	4 bytes	Posición fija y conocida; un byte rojo modificado	Compatible con la cátedra; se lee antes de decodificar; sin redundancia
2. Después del mensaje	Cuatro bits en los primeros bytes libres tras el último bit del mensaje	4 bytes	Posición que depende del largo	El tamaño también está invertido: sin las banderas no se halla el final; hay que guardar el tamaño sin invertir
3. Desplazamiento fijo al final del bloque	Últimos 4 bytes del bloque de píxeles	4 bytes, aunque el mensaje sea corto	Lugar previsible, lejos de la zona del mensaje	Independiente del largo; el mensaje no debe pisarlos
4. LSB de bytes rojos	Cuatro bytes rojos, que LSB1 no usa para el mensaje	Ninguno de verde ni de azul	Hasta cuatro cambios en el rojo, que deja de estar intacto	Posición fija; no altera los grupos, porque los bits 1 y 2 no cambian
5. Campos reservados del encabezado	bfReserved1 y bfReserved2 , bytes 6 a 9 [14]: 32 bits	Ninguno de píxeles	Alta: deben valer cero	Frágil: los editores reescriben el encabezado; este programa no los examina
6. Bytes de relleno de fila	Relleno al final de cada fila si el ancho por 3 no es múltiplo de 4	Ninguno de píxeles, pero no siempre existe (lado.bmp no tiene)	Suele valer cero: otro valor es sospechoso	No siempre hay lugar; la consigna manda usar el relleno como portador
7. Posiciones pseudoaleatorias	Una semilla derivada de la contraseña (PBKDF2) elige 4 posiciones	4 bytes	Baja para quien no tiene la clave	Exige contraseña aun sin cifrado; hay que excluir las posiciones del mensaje
8. No guardarlo	Probar las 16 combinaciones de banderas y quedarse con la que da un encuadre válido	Ninguno	Ninguna marca en la imagen	Hasta 16 decodificaciones; posible ambigüedad
9. Copias redundantes	Tres copias de las banderas en posiciones separadas, decisión por mayoría	12 bytes	Más bytes alterados y un patrón repetido	Tolera el daño de una copia
10. Mapa por bloques	Cuatro banderas por tramo del bloque de píxeles, al inicio de cada tramo	4 bytes por tramo	Banderas dispersas por la imagen	Más ganancia de inversión, pero más costo y complejidad

Cuadro 9: Alternativas para guardar el registro de patrones invertidos

Análisis de las alternativas principales

Después del mensaje. Es el contraste más directo con la elección actual, que el enunciado llama “antes del mensaje”, pero tiene un problema circular. En LSBI el campo de tamaño forma parte del mensaje y, por lo tanto, también está sujeto a la inversión. Para decodificar el tamaño hace falta conocer las banderas, y para ubicar las banderas después del mensaje hace falta conocer el tamaño. Se sale del círculo de dos maneras: guardando el tamaño sin invertir en un lugar fijo (por ejemplo, tratando los primeros 4 bytes del mensaje como una excepción a la inversión), o poniendo el registro en un desplazamiento que no depende del largo. En el primer caso el receptor lee el tamaño, calcula dónde terminan los datos y lee las banderas; en el segundo se vuelve a un lugar fijo, como en la alternativa 3.

En los bytes rojos. Es la opción con mejor relación costo-beneficio dentro del esquema de Majeed y Sulaiman: los bytes rojos no llevan mensaje, de modo que las banderas no le quitan capacidad a verde ni a azul, y los grupos no se alteran, porque los bits 1 y 2 siguen iguales. Tiene la desventaja de romper el rasgo distintivo del método, que deja el canal rojo intacto; aun así, son cuatro bytes de más de trescientos mil, y la huella estadística del canal es despreciable. La elección actual ya es casi esta: tres de las cuatro banderas van en verde y azul, y sólo una en rojo.

Campos reservados del encabezado. Guardar las banderas en `bfReserved1` y `bfReserved2` no gasta ningún byte de píxel y deja intacto el bloque de datos, pero la especificación del formato [14] los reserva y los define en cero. Un valor distinto es una señal obvia para cualquiera que revise el encabezado, y los programas que reescriben el archivo suelen ponerlos a cero, con lo que el registro se perdería sin aviso. Además, este programa no examina esos campos al leer, así que habría que agregar su lectura y escritura explícitas. Es una opción razonable sólo si se controla todo el recorrido del archivo.

Posiciones derivadas de la clave. Si las cuatro posiciones salen de la contraseña, un atacante que no la conoce debe además localizar las banderas. Es la alternativa que más se acerca a un secreto real, porque la ubicación dependería de una clave y no del método. Su límite es que LSBI puede usarse sin cifrado, y entonces no hay contraseña; y que hay que evitar que las posiciones caigan sobre bytes que llevan mensaje.

Sin registro. El receptor puede no necesitar ninguna bandera. Como sólo hay 16 combinaciones posibles de las cuatro banderas, puede decodificar el flujo con cada una y conservar la que produce un encuadre válido: un tamaño dentro de la capacidad, una extensión que empieza con un punto y termina en un byte nulo y, si el mensaje está cifrado, un relleno PKCS5 correcto al descifrar. Un flujo mal decodificado casi nunca pasa esas comprobaciones. El costo es de hasta 16 decodificaciones y una probabilidad pequeña pero no nula de ambigüedad si dos combinaciones dan un encuadre válido; en cambio no gasta capacidad, no deja marca alguna en la imagen y elimina el problema de dónde poner el registro.

Redundancia y mapas por bloques. Repetir las banderas y decidir por mayoría sólo compensa daños en el archivo estego, que LSB no resiste de todos modos. Un mapa por bloques, en cambio, cambia el rendimiento del método: al contar por tramos, cada tramo decide sus inversiones con su propia estadística local y la ganancia total esperada crece con la cantidad de tramos, pero cada tramo paga cuatro bytes de banderas. Habría que medir el punto de equilibrio para cada tamaño de tramo.

Preferencia

Si pudiéramos elegir libremente, consideramos que la mejor opción es no guardar el registro, porque no cuesta capacidad, no deja rastro en la imagen y no crea ninguna posición previsible, a

cambio de una decodificación más lenta y de una ambigüedad muy improbable que se detecta comprobando el encuadre. Como alternativa que sí guarda algo, elegiríamos los cuatro bytes rojos, por no gastar capacidad de verde ni de azul. Ninguna de las dos produce archivos que el programa de la cátedra pueda extraer, porque éste espera las banderas donde manda la consigna. Por eso la implementación sigue lo que la consigna especifica, y las alternativas quedan como un análisis de diseño.

Cuestión IX: Dificultades de implementación

***Enunciado.** ¿Qué dificultades encontraron en la implementación del algoritmo del paper?*

Las dificultades que siguen son las que quedaron registradas en el repositorio durante la implementación, no una reconstrucción de memoria. Cada una nombra el archivo donde puede comprobarse: las notas `docs/LSBI-NOTES.md` (entradas A1 a A10) y `docs/CRYPTO-NOTES.md` (entradas K1 a K8), un caso de `tests/run_tests.sh`, el `Makefile`, el archivo `.gitattributes` o los resúmenes de cada fase. Las ambigüedades del propio documento de Majeed y Sulaiman [1] se analizan en la Cuestión I; aquí se cuenta cómo se enfrentaron al programar.

El algoritmo LSBI

El paper deja sin definir casi todo lo que hace falta para escribir código: en qué orden se recorren los bytes, qué canales llevan datos, dónde se guarda el registro de patrones invertidos, sobre qué bytes se cuentan los cambios, qué pasa en un empate y en qué orden se leen los dos bits del patrón. Con varias lecturas posibles para cada una de esas preguntas, escribir directamente el programa en C habría impedido distinguir un error de lectura de un error de código. Por eso primero se escribió un modelo de referencia en Python (`tools/lsbi.py`) con un interruptor por cada lectura alternativa, y se lo enfrentó a los tres archivos LSBI de la cátedra antes de escribir una línea de C.

La sonda (`python3 tools/lsbi.py probe`) decodifica el vector con la lectura elegida e informa el tamaño leído, si el encuadre es válido y si volver a insertar el mensaje reproduce el archivo. El cuadro 10 resume lo que dio cada lectura sobre `Ejemplo/lado.bmp` y el vector correspondiente. El tamaño correcto es 44 886 bytes (la imagen PNG oculta). Seis lecturas se descartan y dos no pueden decidirse con los vectores.

Una lección de este trabajo está en la fila del orden de los bits: `reembed_identical=yes no` prueba que la decodificación sea correcta. Una decodificación equivocada también se reinserta sin pérdida, porque codificar y decodificar con la misma regla equivocada es una identidad. Lo que discrimina es el tamaño decodificado y el hash del PNG extraído (nota A3 y resumen de la fase 2). Por eso las notas registran en cada caso el tamaño decodificado y el `sha256` de la imagen extraída.

Lo que queda sin decidir es importante para los archivos de estegoanálisis: la regla de empate (nota A6), el orden de los dígitos del patrón (A4, que sólo importaría si las banderas de 01 y 10 difirieran, cosa que ninguno de los tres vectores muestra), la capacidad máxima y el texto del mensaje de error (A8) y el conteo del salto del rojo cuando hay relleno de filas (A9, porque `lado.bmp` mide 640 píxeles de ancho y no tiene relleno). Las cuatro decisiones están marcadas `flagged-unverified` en las notas, para revisarlas primero si aparece un archivo de la cátedra que las contradiga.

Lectura alternativa	Interruptor de la sonda	Resultado observado	Conclusión
Los bytes rojos también llevan mensaje	<code>-use-red</code>	tamaño 658, encuadre de texto cifrado, la reinserción no coincide	Descartada
El salto del rojo se cuenta desde el byte 4	<code>-red-relative</code>	tamaño 109 078, encuadre inválido, la reinserción no coincide	Descartada
Bandera del patrón p en el byte $3 - p$	<code>-flag-order reversed</code>	sobre el vector AES-256 (banderas 0001): tamaño 4 294 922 424, encuadre inválido, la reinserción no coincide	Descartada
Bits de cada byte del mensaje del menos al más significativo	<code>-bit-order lsb</code>	tamaño 62 826, encuadre de texto cifrado (pero la reinserción <i>sí</i> coincide)	Descartada
Los cambios se cuentan también en los bytes rojos	<code>-count-red</code>	banderas recalculadas 0000 contra 1111; la reinserción no coincide	Descartada
Una bandera en 0 significa patrón invertido	<code>-flag-zero-inverts</code>	tamaño 4 294 922 409, encuadre inválido, la reinserción no coincide	Descartada
Un empate de cambios invierte el patrón	<code>-tie invert</code>	la reinserción coincide igual	Indecidible
Los dos dígitos del patrón se leen al revés (01 y 10)	<code>-pattern-bits 12</code>	la reinserción coincide igual en los tres vectores	Indecidible

Cuadro 10: Lecturas alternativas de LSBI probadas contra los vectores de la cátedra (entradas A1 a A6 de docs/LSBI-NOTES.md)

Encriptación

Una errata en el archivo de ejemplo. Ejemplo/README.txt lista, para el archivo ladoLSB1aes128cbc.bmp, el vector de inicialización 212420edc583a686a94d19a3497363a2, y una clave derivada rotulada “32 bytes” que en realidad tiene 32 dígitos hexadecimales, es decir, 16 bytes. Con la derivación descrita en la consigna, `python3 tools/cryptoref.py kdf aes128 margarita` da la misma clave (03db0a157acfe8de523760aa731d8122) pero el vector b25f8d99f3173ec0b52849f459a4c20d; el de la lista es el de AES-256. Con el vector de la lista, la lectura del archivo falla con `decrypted size field says 2474312226 bytes but the plaintext has only 44895 bytes`, porque el primer bloque CBC sale corrupto (sólo cambia el IV, así que el relleno del último bloque sigue siendo válido). Con el vector derivado se recupera el PNG de 44 886 bytes. El archivo mismo decidió cuál era el error (nota K3, casos `cryptoref-kdf-aes128` y `cryptoref-readme-aes128-iv-erratum`).

Una sola derivación, primero la clave. La consigna no dice cómo se separan clave y vector según el algoritmo. Los tres vectores de la cátedra obedecen una sola llamada a PBKDF2 [9] que produce `long_clave + long_iv` bytes, con la clave primero (nota K2). Esa regla es la que fija AES-192, que no tiene archivo de ejemplo (nota K4, sin verificar contra un vector de la cátedra: el oráculo independiente aplica la misma regla, de modo que prueba consistencia y no corrección).

CFB de 8 bits y OFB de 128 bits. Los nombres de la consigna deben traducirse con cuidado a los de OpenSSL [16]: CFB es `EVP_*_cfb8` en todos los algoritmos y OFB es el `EVP_*_ofb` de bloque completo, con lo que el OFB de 3DES tiene forzosamente realimentación de 64 bits [10]. El vector de 3DES sólo se descifra con `cfb8` (nota K6).

Relleno y capacidad. Sólo ECB y CBC rellenan (PKCS5). El programa necesita conocer la longitud del texto cifrado *antes* de comprobar la capacidad del portador, así que la calcula: es la del texto plano redondeada hacia arriba al siguiente múltiplo del bloque (más un bloque si ya era múltiplo) para ECB y CBC, y la del texto plano para CFB y OFB. Se midió contra el

oráculo para las 16 combinaciones (nota K5).

Contraseña equivocada sin código de autenticación. El formato no tiene MAC. Una contraseña, algoritmo o modo equivocados se detectan por el relleno PKCS5 (ECB y CBC) y por una validación estricta del encuadre del texto plano (tamaño coherente, un NUL final, extensión imprimible), de modo que casi siempre hay un error claro (código de salida 2) y ningún archivo de salida. Es posible, aunque muy improbable, que una contraseña equivocada pase ambos controles y produzca basura. Es una debilidad del formato, no del código, y se registró en la sección de modelo de amenazas de docs/CRYPTO-NOTES.md.

Sin -pass no hay cifrado. La consigna dice que sin contraseña se hace esteganografía sin cifrar. Si el usuario da -a o -m pero no -pass, el programa ignora esas opciones, y como el silencio sería confuso imprime en el error estándar `stegobmp: note: -a/-m ignored because no -pass was given (no encryption)` (nota K8).

Formato del mensaje y portador

El tamaño va en cuatro bytes *big-endian*, seguido de los datos, de la extensión (un punto y sus letras) y de un byte NUL; con cifrado se antepone el tamaño del texto cifrado. Como cualquier archivo estego puede haber sido alterado, el tamaño leído se acota con la capacidad restante del portador antes de reservar memoria, y el texto plano descifrado se valida por completo antes de escribir nada (casos `decrypt-hostile-size-*` y `cap-enc-sparse-5GiB`, resumen 03-02).

El portador también trae detalles. El bloque de píxeles se usa completo, con los bytes de relleno de fila incluidos y en el orden del archivo (no en el orden de la imagen); las filas van de abajo hacia arriba, aunque el programa admite alturas negativas; el encabezado y cualquier byte posterior al bloque se copian sin cambios. Todas las salidas se escriben primero en un archivo temporal del mismo directorio y luego se renombran sobre el destino (`rename`), de modo que un error nunca deja un archivo a medias, incluso si `-out` coincide con `-p` (documento SKELETON.md de la fase 1).

La consigna y los archivos de ejemplo

Los comandos de ejemplo de la consigna [2] no se pueden copiar y pegar. Están escritos con un guion largo (-) en lugar del guion común y con comillas tipográficas (“mensaje1.txt”): el texto extraído del PDF contiene `-in`, `-p`, `-steg` y `-a`. Tecleado tal cual, `./stegobmp -embed -in x -p Ejemplo/lado.bmp -out y.bmp -steg LSB1` termina con código de salida 1 y el mensaje `stegobmp: error: unknown parameter '-in'` seguido de las líneas de uso. Se optó por respetar la sintaxis exacta del enunciado y no aceptar el guion largo, porque la consigna exige respetar esa sintaxis. El ejemplo de extracción, además, escribe `"imagenmas1 .bmp"` con un espacio antes de la extensión, que no es el nombre del archivo que se generó en el ejemplo anterior.

Otras incoherencias menores: el método se llama “LSB Enhanced” en la sección 5.1 y “LSB Improved” en el resto de la consigna, aunque el valor de `-steg` es siempre `LSB1`; el ejemplo 2 llama “imagen” a un archivo `.txt`; los indicadores de patrones se guardan con los tres canales R, G y B mientras que el mensaje salta el rojo, cosa que sólo dice el paper (nota A2 y punto 16 de las incoherencias listadas en docs/LSBI-NOTES.md). Por último, `Ejemplo/README.txt` no está en UTF-8: el comando `file` lo identifica como “ISO-8859 text, with CRLF line terminators”, así que sus letras acentuadas aparecen como caracteres de reemplazo en una terminal UTF-8.

Compilación y portabilidad

La consigna exige que el programa compile y corra en Ubuntu 22.04.3 y en pampero, dos entornos con versiones distintas de OpenSSL (de la 1.0.2 a la 3.x en los sistemas que podían aparecer), cuya interfaz de EVP cambió de forma incompatible entre ellas (por ejemplo, la creación y destrucción de contextos). El código se limita a llamadas comunes a 1.0.2, 1.1.1 y 3.x, y el caso `build-openssl-api-allowlist` de `make test` falla si aparece un identificador de OpenSSL fuera de una lista permitida, un encabezado distinto de `evp.h`, `crypto.h` o `err.h`, o una biblioteca de enlace distinta de `libcrypto`.

Se verificó la compilación limpia y las pruebas en pampero (OpenSSL 3.6.3) y en Ubuntu 24.04 bajo WSL2 (OpenSSL 3.0.13, gcc 13.3). **El entorno exacto que nombra la consigna, Ubuntu 22.04.3, no se probó:** la distribución no estaba instalada en la máquina de desarrollo y se renunció a esa verificación; lo que da confianza en su lugar es la lista de identificadores permitidos y las dos pruebas en entornos reales.

Tres problemas concretos de construcción quedaron documentados. Primero, `CC ?= gcc` en el `Makefile` no tiene efecto, porque `make` ya define `CC` como `cc`; se corrigió con `ifeq ($(origin CC),default)`, que conserva los cambios hechos desde la línea de comandos (resumen 03-03). Segundo, la dependencia de enlace se comprueba con las entradas `NEEDED` de `readelf` y no con `ldd`, para que las dependencias transitivas de `libcrypto` en pampero no cuenten como dependencias del programa. Tercero, una copia de trabajo en Windows con `core.autocrlf=true` convierte los finales de línea en CRLF, con lo que `bash` y `make` fallan bajo WSL; se resolvió con `.gitattributes`, que fuerza `eol=lf` en `*.c`, `*.h`, `*.sh`, `*.py` y `Makefile` y declara binarios los `.bmp`, `.png` y `.pdf` para que los vectores nunca se conviertan como texto. Hay además fricciones propias de WSL: `/tmp` no se comparte entre invocaciones separadas de `wsl` (las comparaciones antes y después hubo que hacerlas dentro de una misma llamada) y, sobre el disco NTFS, `make` a veces avisa `Clock skew detected`.

Cómo se verificó

`make test` ejecuta 242 casos (SUMMARY: 242 passed, 0 failed), y `make test-tools` ejecuta 42 casos (SUMMARY: 42 passed, 0 failed) de las herramientas de análisis. Incluyen: la extracción de los seis vectores de la cátedra y su reinserción byte a byte (cada archivo se reproduce exactamente al insertar de nuevo el mismo mensaje con la misma clave); la matriz de métodos por algoritmos y modos, con un recorrido completo por LSB1, LSB4 y LSBI con y sin cifrado; casos de contraseña, algoritmo y modo equivocados, de datos dañados y de tamaños hostiles; corridas con AddressSanitizer y UBSan; y la ejecución textual de los ejemplos del README por `tests/check_readme.sh`, para que la documentación no pueda divergir del programa.

El descifrado se compara con un oráculo independiente, `tools/cryptoref.py`, que usa `hashlib.pbkdf2_hmac` de Python y la línea de comandos de `openssl`, y no comparte ningún código con la implementación en C. Esa independencia es la que permitió encontrar la errata del archivo de ejemplo. Los límites de la verificación están dichos arriba: AES-192 y varios modos (todas las combinaciones ECB, AES en CFB de 8 bits y 3DES en OFB) sólo están contrastados con el oráculo y no con archivos de la cátedra, y los cuatro puntos de LSBI sin decidir se validarán cuando lleguen los archivos de estegoanálisis.

Cuestión X: Mejoras y extensiones futuras

Enunciado. *¿Qué mejoras o futuras extensiones harías al programa stegobmp?*

Todo lo que sigue es una propuesta. Ninguna de estas mejoras está implementada en `stegobmp`: el programa entregado sigue al pie de la letra el formato de la cátedra [2], porque su criterio de aceptación es extraer los archivos de la cátedra y producir archivos que la cátedra pueda extraer. Por eso, para cada propuesta se indica si rompería la compatibilidad byte a byte con ese formato. Cuando lo rompe, la única forma honesta de incorporarla sería como un modo opcional, con su propia opción de línea de comandos, dejando intacto el comportamiento por omisión.

La lista está ordenada por la prioridad que le daría el autor: primero lo que corrige debilidades reales de seguridad, después lo que mejora la indetectabilidad, después lo que amplía el alcance del programa y, al final, lo que mejora el proceso de desarrollo. Las debilidades de las primeras propuestas no son defectos de implementación sino consecuencias de decisiones fijadas por la consigna (sal fija, modo ECB disponible, ausencia de código de autenticación), registradas como tales en `docs/CRYPTO-NOTES.md`.

1. **Cifrado autenticado.** *Motivación:* hoy no hay ningún código de autenticación. Una contraseña equivocada se detecta sólo porque el relleno PKCS5 (modos ECB y CBC) o el encuadre del mensaje resultan inválidos, y eso deja una probabilidad pequeña pero no nula de aceptar una contraseña incorrecta y entregar basura; además, cualquiera puede modificar bits del archivo estego en un modo de flujo (CFB, OFB) y el receptor no lo notará. *Boceto:* cifrar con AES en modo GCM [11] y guardar la etiqueta de autenticación junto al texto cifrado, o bien cifrar y luego calcular un HMAC con una clave derivada por separado (*encrypt-then-MAC*). En ambos casos una contraseña equivocada o un archivo alterado se rechazan siempre. *Costo:* 16 a 32 bytes más de carga útil y una segunda derivación de clave. *Compatibilidad:* rompe el formato de la cátedra, que no tiene lugar para la etiqueta; sería un modo opcional (por ejemplo un valor nuevo de `-m`).
2. **Sal aleatoria y una derivación de clave más costosa.** *Motivación:* la sal de ocho bytes en cero y las 10 000 iteraciones de PBKDF2 [9] hacen que dos mensajes con la misma contraseña usen la misma clave y el mismo vector de inicialización, y permiten ataques de diccionario con tablas precalculadas; 10 000 iteraciones es poco frente a lo que hoy se recomienda [12]. *Boceto:* generar una sal aleatoria por mensaje y guardarla junto a la carga (por ejemplo, 16 bytes antes del tamaño cifrado), subir mucho el número de iteraciones o reemplazar PBKDF2 por Argon2 [13], que además exige memoria. *Costo:* 16 bytes de capacidad, tiempo de cómputo por ejecución y, para Argon2, una dependencia que no está en la biblioteca `libcrypto` de todas las versiones de OpenSSL. *Compatibilidad:* rompe el formato de la cátedra (que fija la sal y el número de iteraciones); modo opcional.
3. **Leer la contraseña sin dejarla en la línea de comandos.** *Motivación:* con `-pass` la contraseña queda visible en la lista de procesos y en el historial del intérprete de comandos, algo que ya advierte el README. *Boceto:* si `-pass` se da sin valor (o con una opción aparte), pedir la contraseña en la terminal con el eco desactivado, o leerla de un archivo o de una variable de entorno. *Costo:* código para manejar la terminal, que difiere entre plataformas. *Compatibilidad:* no afecta el formato de los archivos, pero cambia la sintaxis de la línea de comandos que la consigna exige, así que debería ser una opción adicional y no reemplazar a `-pass`.
4. **Orden de inserción que depende de la clave.** *Motivación:* hoy los bits se escriben siempre de forma secuencial desde el primer byte del bloque de píxeles, de modo que quien

conozca el método sabe dónde buscar; esta regularidad es la que aprovechan los ataques estadísticos sobre el prefijo de la imagen. *Boceto*: derivar de la contraseña un generador pseudoaleatorio que elija las posiciones, siguiendo la idea de esteganografía con clave de Sharp [7], y repartir así el mensaje por toda la imagen. *Costo*: el mensaje corto ya no deja un rastro localizado, pero se pierde la lectura secuencial (más lento) y hace falta una contraseña incluso sin cifrado. *Compatibilidad*: rompe el formato; modo opcional.

5. **Emparejamiento LSB (*LSB matching*, ± 1) en lugar de reemplazo.** *Motivación*: el reemplazo de LSB iguala los contadores de cada par de valores ($2k, 2k + 1$), lo que el ataque de χ^2 de Westfeld y Pfitzmann [4] y el análisis RS [5] detectan (lo medimos en las Cuestiones II y VII). *Boceto*: cuando el LSB del byte no coincide con el bit del mensaje, sumar o restar uno al azar en lugar de forzar el bit [7]; el receptor sigue leyendo el LSB. *Costo*: hay que cuidar los extremos (0 y 255) y el cambio puede tocar bits superiores; aun así se resiste mejor a esos dos ataques concretos, no a todos. *Compatibilidad*: el archivo estego resultante se extrae con el programa actual (el LSB es el mismo), pero no coincide byte a byte con el que produciría la cátedra, y la extracción de archivos de la cátedra no cambia.
6. **Inserción adaptativa.** *Motivación*: modificar por igual todas las regiones deja huellas en las zonas lisas, donde el ruido natural es bajo [6]. *Boceto*: calcular un mapa de textura (por ejemplo, la variación local de los píxeles vecinos que no se modifican) y usar de preferencia las zonas ruidosas, además de limitar la carga a una fracción de la capacidad. *Costo*: el receptor debe poder reconstruir el mismo mapa, lo que obliga a que el criterio dependa sólo de bits que la inserción no toque; menor capacidad efectiva. *Compatibilidad*: rompe el formato; modo opcional.
7. **Modo de análisis dentro del programa.** *Motivación*: la consigna (sección 3) espera que se pueda también estegoanalizar un archivo, pero *stegobmp* sólo oculta y extrae; el análisis lo hacen scripts aparte del repositorio. *Boceto*: agregar opciones como `-analyze` y `-capacity` que reutilicen las ideas de `tools/bmpdiff.py` (diferencias entre portador y estego), `tools/lsbstats.py` (estadísticos de LSB y χ^2 por canal) y `tools/sigsearch.py` (búsqueda de firmas de archivos conocidos en el bloque de píxeles y en los datos agregados). *Costo*: código nuevo en C y más superficie que probar. *Compatibilidad*: no afecta el formato; sólo agrega opciones.
8. **Más portadores.** *Motivación*: hoy se aceptan sólo BMP de versión 3 con 24 bits y sin compresión (con alturas positivas o negativas); el resto se rechaza con un mensaje. *Boceto*: admitir encabezados V4 y V5, píxeles de 32 bits (saltando el canal alfa) y formatos sin pérdida como PNG, previa descompresión con una biblioteca. *Costo*: más código de análisis del formato (con más riesgo de errores con archivos hostiles) y una dependencia nueva para PNG, que no se puede asumir presente en pampero. *Compatibilidad*: no afecta a los BMP de 24 bits.
9. **Comprimir antes de cifrar.** *Motivación*: caben archivos más grandes, y un mensaje comprimido tiene mayor entropía, que se parece más al ruido de la imagen. *Boceto*: comprimir con DEFLATE antes de cifrar y marcar el hecho en el encabezado; se debe hacer antes de cifrar, porque un texto cifrado ya no se comprime. *Costo*: una biblioteca de compresión y un indicador que hoy no existe en el encuadre. *Compatibilidad*: rompe el formato; modo opcional.
10. **Variantes de LSBI.** *Motivación*: la Cuestión VII muestra que la ganancia de LSBI es pequeña. *Boceto*: grupos de patrones más finos (tres bits), banderas por bloque de imagen (véanse las alternativas de la Cuestión VIII), banderas también en el canal rojo, y la misma idea de inversión aplicada a LSB4. *Costo*: más banderas y más capacidad consumida, para

una ganancia que habría que medir con las herramientas del repositorio. *Compatibilidad:* rompe el formato de LSBI de la cátedra; modo opcional.

11. **Control de integridad para mensajes sin cifrar.** *Motivación:* extraer con un método equivocado (por ejemplo `-steg LSB1` sobre un archivo `LSB4`) sin contraseña puede producir un archivo sin sentido en lugar de un error, porque el único control es que el tamaño sea posible y que el encuadre de la extensión sea válido. *Boceto:* guardar un CRC de 32 bits del contenido al final del mensaje y comprobarlo al extraer. *Costo:* 4 bytes. *Compatibilidad:* rompe el formato de la cátedra; modo opcional.
12. **Integración continua.** *Motivación:* el comportamiento se verificó a mano en Ubuntu 24.04 (WSL2) y en pampero, pero no en la Ubuntu 22.04.3 que nombra la consigna, y las versiones de OpenSSL varían (3.0.13 y 3.6.3 en los entornos probados). *Boceto:* un flujo automático que compile y ejecute `make test`, `make test-tools` y la compilación limpia del README en Ubuntu 22.04, con las versiones de OpenSSL de pampero. *Costo:* configuración y mantenimiento del flujo. *Compatibilidad:* no afecta al programa.

Las tres primeras que haría el autor serían la 1, la 2 y la 7. El cifrado autenticado con sal aleatoria cierra las dos debilidades de seguridad reales del formato actual, la aceptación de contraseñas equivocadas y la falta de detección de alteraciones, con un costo de unos pocos bytes. El modo de análisis integrado responde a un pedido explícito de la consigna que hoy queda fuera del programa, y no toca el formato. La propuesta 5, el emparejamiento LSB, la seguiría de cerca porque es la única que mejora la indetectabilidad sin cambiar cómo se extrae. Las demás dependen de decidir primero cuánta compatibilidad con la cátedra se está dispuesto a sacrificar.

Referencias

- [1] M. A. Majeed and R. Sulaiman, “An improved LSB image steganography technique using bit-inverse in 24 bit colour image”, *Journal of Theoretical and Applied Information Technology*, vol. 80, no. 2, pp. 342–348, 2015.
- [2] Cátedra de Criptografía y Seguridad, ITBA, “Trabajo Práctico de Implementación: Esteganografía”, 2026.
- [3] N. F. Johnson and S. Jajodia, “Exploring steganography: Seeing the unseen”, *IEEE Computer*, vol. 31, no. 2, pp. 26–34, 1998.
- [4] A. Westfeld and A. Pfitzmann, “Attacks on steganographic systems”, *Information Hiding 1999*, LNCS 1768, pp. 61–76, Springer, 2000.
- [5] J. Fridrich, M. Goljan and R. Du, “Reliable detection of LSB steganography in color and grayscale images”, *Proc. ACM Workshop on Multimedia and Security*, pp. 27–30, 2001.
- [6] J. Fridrich, *Steganography in Digital Media: Principles, Algorithms, and Applications*, Cambridge University Press, 2009.
- [7] T. Sharp, “An implementation of key-based digital signal steganography”, *Information Hiding 2001*, LNCS 2137, pp. 13–26, Springer, 2001.
- [8] A. Kerckhoffs, “La cryptographie militaire”, *Journal des sciences militaires*, vol. IX, pp. 5–38, 1883.
- [9] K. Moriarty, B. Kaliski and A. Rusch, “PKCS #5: Password-Based Cryptography Specification Version 2.1”, RFC 8018, 2017.

- [10] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Methods and Techniques”, NIST SP 800-38A, 2001.
- [11] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC”, NIST SP 800-38D, 2007.
- [12] M. Sönmez Turan, E. Barker, W. Burr and L. Chen, “Recommendation for Password-Based Key Derivation, Part 1: Storage Applications”, NIST SP 800-132, 2010.
- [13] A. Biryukov, D. Dinu, D. Khovratovich and S. Josefsson, “Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications”, RFC 9106, 2021.
- [14] Microsoft, “BITMAPFILEHEADER structure (wingdi.h)”, documentación de la API Win32, enlace indicado en el enunciado del TP.
- [15] R. Gómez Cárdenas, “Esteganografía”, <http://www.cryptomex.org/SlidesCripto/Estegano.pdf>.
- [16] OpenSSL Project, “EVP_EncryptInit” (página de manual), <https://www.openssl.org/docs/>.